Agentic Self-Correction in LLaMA3: Enhancing E-Commerce Sentiment Analysis through Multi-Agent Reflection

Baseline (Phase 1) + Proposed Agentic Framework (Phase 2)

Document revised March 2026. Matches `pes-mtech-project` Colab notebooks, eval methodology in §10, and Hugging Face `datasets` 3.x (no dataset loading scripts for Amazon Reviews 2023 — see §4.2).

1. Abstract

Automated Sentiment Analysis (ASA) plays a critical role in modern e-commerce systems by enabling businesses to analyze large volumes of customer feedback and identify trends related to product quality and user satisfaction. Recent advancements in Large Language Models (LLMs), particularly LLaMA-3, have significantly improved the performance of sentiment classification tasks due to their strong contextual understanding and language reasoning capabilities.

However, these models typically operate as **static, text-only classifiers**: they generate predictions in a single inference step without performing factual verification or reflective reasoning. This limitation often leads to **sentiment hallucinations**, where incorrect user claims (e.g. “this camera isn’t waterproof”) are interpreted as legitimate product defects even when product specifications state otherwise. Additionally, single-pass models struggle with complex linguistic phenomena such as sarcasm (“Amazing, it broke in two days”), mixed sentiment, and ambiguous expressions.

This project proposes a **multimodal Agentic AI framework** for sentiment analysis that integrates **review text, product images, and product metadata** to improve classification reliability. The system introduces a **Self-Correction Agent** that acts as a critic, continuously auditing the predictions generated by the primary sentiment analysis model. A **Retrieval-Augmented Generation (RAG)** pipeline retrieves product specifications from a vector database (LanceDB) to verify whether claims in the review are technically valid. By transitioning from a passive classification system to a **Verifier-in-the-Loop** architecture, the proposed framework produces sentiment predictions that are both **linguistically accurate and factually grounded**, enabling e-commerce platforms to obtain more reliable insights from customer feedback.

2. Problem Statement

Current AI models used for sentiment analysis are excellent at understanding language but struggle with **factual accuracy**. Because models like LLaMA-3 typically process a review in a single, "text-only" step, they lack the ability to verify if a customer's claims are actually true. This leads to three primary issues:

- **Sentiment hallucinations:** The AI often mistakes a user's incorrect claim (e.g. "this watch isn't waterproof") for a legitimate product defect, even if the product specifications prove otherwise. The model has no access to spec sheets and thus cannot distinguish between a real defect and a user error or wrong expectation.
- **Context blindness:** By ignoring product images and technical metadata, the AI misses vital evidence that could confirm or refute a user's complaint. For example, a user might write a neutral review but upload a photo of a shattered screen; a text-only system is blind to this visual evidence and produces an incomplete or misleading sentiment profile.
- **Linguistic errors:** Without a "double-check" mechanism, models frequently misinterpret complex human emotions like sarcasm or mixed feelings. A phrase such as "Amazing, it broke in two days" may be scored positively if the model focuses on the word "amazing" without a critic to trigger re-evaluation.

The gap: There is a critical need for a system that doesn't just "read" sentiment but **verifies** it against product facts and other modalities. This research proposes an Agentic AI framework that uses a Verifier-in-the-Loop approach to ensure sentiment predictions are both linguistically smart and factually grounded.

3. Research Gap

3.1 Baseline study: supervised fine-tuning with LLaMA-3 (Wang et al., 2025)

The baseline establishes a state-of-the-art benchmark for sentiment classification using LLMs. **Model:** LLaMA-3-8B as the backbone, enhanced with Low-Rank Adaptation (LoRA) so that only low-rank matrices (A and B) are updated while the primary model weights stay frozen. **Data strategy:** (1) **DQC (Data Quality Control):** TextBlob is used to filter out "noisy" reviews where the text sentiment contradicts the numerical rating. (2) **VGST (Variant Greedy Search Technique):** A diversity-maximizing sampling method ensures the training set (e.g. 2000 samples) represents a wide array of linguistic patterns. **Reasoning:** One-Shot Learning combined with Zero-Shot Chain-of-Thought (CoT) prompts (e.g. "Let's take it one step at a time") guide the model through the classification logic.

Paper-reported performance: Accuracy 92.6%, Precision 95.3%, F1 93.3%, outperforming traditional baselines such as SVM and XLNet.

Our Phase 1 implementation (`colab/llama_sentiment_baseline_train.ipynb`): LLaMA-3-8B + **PEFT/LoRA** (defaults often stronger than paper Table 1: e.g. r=16, α=128, 5 epochs, `max_samples`=4000). **Typical Colab eval** after pipeline/eval fixes is on the order of **~66% accuracy** and **~68% weighted F1** (§10); paper ~92% remains the reference under their full setup — we target sound methodology and the same metric types, not bit-for-bit reproduction.

Despite strong paper numbers, the baseline is fundamentally a **passive, single-pass classifier** with no factual verification or critic — which motivates Phase 2.

3.2 The research gap: from passive to agentic

Three specific gaps separate the baseline from the proposed agentic system:

- **Absence of factual grounding (the "gullibility" gap):** The baseline assumes the user's text is always factually correct. If a user reviews a waterproof camera and claims "it broke in the rain," the baseline will mark this as a negative (e.g. Rating 1). The model doesn't know the camera is rated for 30 m underwater and cannot verify whether the claim is a product defect or user error.
- **Lack of autonomous auditing (the "critic" gap):** The baseline relies on a single "Analyst" (the fine-tuned LLaMA-3). There is no Critic Agent to perform a logical audit. If the model sees "amazing" in a sarcastic sentence like "Amazing, it broke in two days," it might still lean toward a positive score because there is no self-correction loop to catch the irony.
- **Modality isolation (the "visual" gap):** The baseline processes text-only data. In e-commerce, a picture can be worth a thousand words: a user might write a neutral review but upload a photo of a shattered screen. The baseline is blind to this visual evidence, leading to an incomplete sentiment profile.

4. Datasets

4.1 Baseline dataset (Kaggle)

Source: `arhamrumi/amazon-product-reviews` on Kaggle. **Size:** approximately 500,000 product reviews. **Labels:** rating scale from 1 to 5 (1 = most negative, 5 = most positive). **Fields per record:** review text, rating, review summary, product category.

Issues: The raw dataset contains duplicates, inconsistent sentiment–rating pairs (e.g. very negative text with a 5-star rating), and a strong bias toward positive reviews. Therefore the data is processed with quality control (TextBlob DQC) and balancing (stratified sampling, oversample neutral) before training.

4.2 Proposed dataset (Amazon Reviews 2023)

Source: Amazon Reviews 2023, developed by UCSD McAuley Lab (amazon-reviews-2023.github.io). **Scale:** 570 million reviews, 48 million products, 33 product categories. **Components:**

- **Review data:** review text, rating score, user ID, timestamp.
- **Product metadata:** product title, description, category, price, technical specifications.
- **Product images:** product photographs, multiple views.

This dataset enables **multimodal reasoning**: the proposed architecture can use text, images, and specs together for verification and dissonance calculation, which the baseline dataset cannot support.

Loading on Hugging Face (`datasets` ≥ 3): Hub dataset **Python scripts are disabled**. Use `load_dataset("json", data_files=…)` for reviews at `raw/review_categories/{Category}.jsonl` (e.g. `All_Beauty.jsonl` for config `raw_review_All_Beauty`) and `load_dataset("parquet", data_files=…)` for all shards under `raw_meta_{Category}/*.parquet`. Do **not** use `trust_remote_code` for this dataset. Demo notebooks: `phase1_vs_phase2_amazon2023_presentation.ipynb`, `phase2_agentic_vs_phase1_amazon2023.ipynb`.

5. Baseline Data Flow (High-Level)

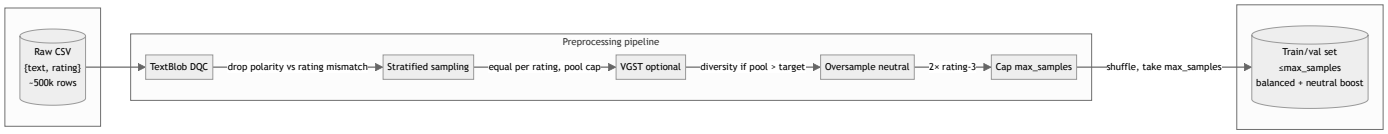
```
Raw CSV → TextBlob (DQC) → Stratified → VGST → Oversample Neutral → Cap max_samples → Train/Val Set
```

Input: Raw data is a list of `{"text": "<review>", "rating": 1..5}` from the CSV; the full dataset can be ~500k rows with ratings often highly imbalanced (many 4–5, fewer 1–2–3). **Output:** A balanced pool (stratified, often `stratified_max_total=10000` before VGST), then capped to `max_samples` (default **4000** in the current notebook to match paper §3; paper Table 1 also lists 2000). Neutral (rating 3) is oversampled before the cap. Order: TextBlob → stratified → VGST (optional) → oversample neutral → cap. Implemented in `apply_paper_preprocessing()` in `colab/llama_sentiment_baseline_train.ipynb`.

5.1 Mermaid: Linear data flow

Each block is one stage of the preprocessing pipeline; data flows left to right until the final train/validation set.

5.2 Mermaid: Preprocessing pipeline with subgraphs



Input: raw data. Pipeline: each step applies its rule (e.g. drop inconsistent polarity, balanced per rating). Output: final train/val set capped at `max_samples` (default 4000).

6. Data Preprocessing Steps (Detail)

Step 1: TextBlob (Data Quality Check – DQC)

Purpose: Remove reviews where the **text sentiment** disagrees with the **human rating**, so the model is not trained on noisy or mislabeled examples. This is the paper's "DQC" (data quality consistency).

How it works: For each row, compute the **TextBlob polarity** of the review text: a float in [–1, 1] (negative → positive). Conceptually, negative polarity aligns with ratings 1–2, neutral with 3, positive with 4–5. **Discard** the row if: (a) `polarity < 0` and `rating > 3` (text is negative but label says positive/neutral), or (b) `polarity > 0` and `rating < 3` (text is positive but label says negative/neutral). If polarity is exactly 0, keep the row regardless of rating. If TextBlob throws (e.g. empty text), keep the row.

```
if pol < 0 and rating > 3: drop if pol > 0 and rating < 3: drop else: keep
```

Effect: Fewer rows; only rows where the written sentiment is consistent with the 1–5 label remain.

Step 2: Stratified sampling (equal per rating)

Purpose: Balance the dataset so each **rating 1, 2, 3, 4, 5** has the **same number** of samples. This reduces class imbalance and avoids the model biasing toward the majority class (often 4–5). The paper calls for "an equal number of samples for each distinct rating level."

How it works: Group all rows by `rating` into five buckets. With `max_total` from `stratified_max_total` or `max_samples` (often 5000–10000 before cap), take equal counts per rating, then shuffle. The notebook default uses a larger stratified pool (e.g. 10000) before VGST/cap.

Effect: A balanced set: same number of 1s, 2s, 3s, 4s, and 5s per tier up to the stratified cap.

Step 3: VGST (Variant Greedy Search Technique)

Purpose: When there are **more** rows than the target size (e.g. in other setups stratified might yield 10k rows), VGST selects a **diverse subset** by favoring reviews that add the most **new tokens** not yet seen in the selected set. The final training set is both size-limited and **lexically diverse** (paper's "DDC" – data diversity and complexity). The full Greedy Search Technique would be too expensive; VGST uses `batch_size` and `wishlist_len` to trade efficiency for quality.

How it works: Set `target_size = min(len(rows), max_samples)`. If you already have 2000 rows from stratified, `target_size = 2000` and VGST is **skipped**. When VGST runs:

1. Start with an empty **sample** and an empty set **current_tokens** (token IDs already covered).
2. Until the sample has `target_size` items: (a) Build a **wishlist** of `wishlist_len` candidates (default 10): randomly draw a batch of `batch_size` rows (default 32); for each row, tokenize the review and count how many **new** token IDs it adds vs `current_tokens`; keep the row that adds the most new tokens and add it to the wishlist. (b) From the wishlist, take the best row (max new tokens) and add it to the sample; add its tokens to `current_tokens`.
3. If at some point no row adds new tokens, fill the rest with random rows.

Parameters: `batch_size` = 32 (candidates per wishlist slot); `wishlist_len` = 10 (candidates per round before picking the best).

Effect: A subset of size equal to the cap that tends to maximize vocabulary diversity. In our pipeline, VGST runs when the stratified pool is larger than the final `max_samples` target.

Step 4: Oversample neutral (rating 3)

Purpose: Rating **3 (neutral)** is harder to learn (subtle, mixed sentiment). The paper "deliberately increased their representation" so the model sees more neutral examples.

How it works: Split rows into **neutral** (rating == 3) and **others** (1, 2, 4, 5). Compute `n_extra = (multiplier − 1) × len(neutral)`; default `multiplier = 2.0` so `n_extra = len(neutral)`. Draw `n_extra` rows **with replacement** from the neutral list. Final set = others + neutral + extra, then shuffle.

Effect: Roughly twice as many rating-3 examples; total number of rows can exceed 2000 (e.g. 2000 + 400 if there were 400 neutrals).

Step 5: Cap at max_samples

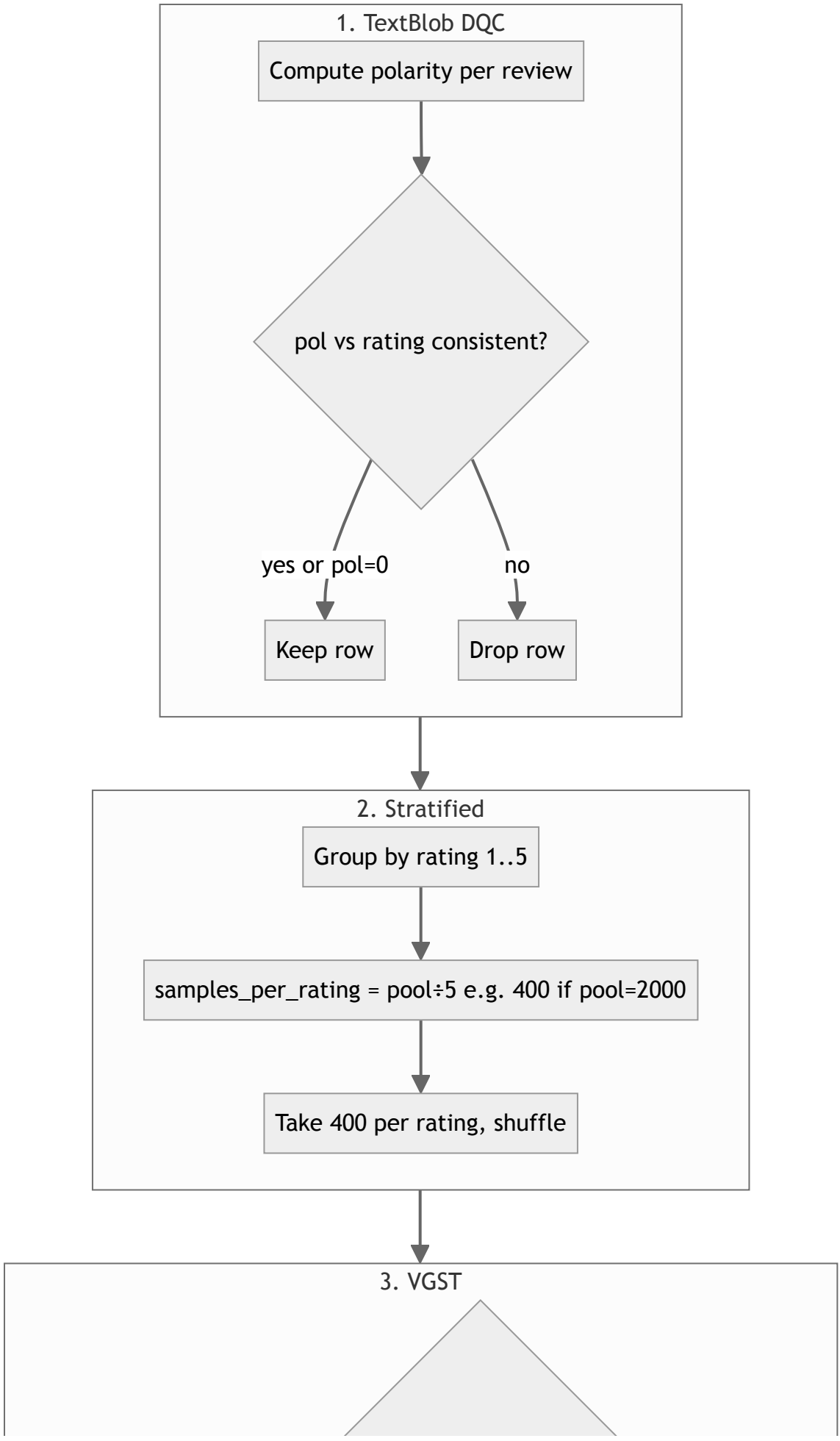
Purpose: Paper Table 1 lists **2000** samples; paper §3 also uses **4000** training reviews. The notebook default is `max_samples=4000`. After oversample neutral, shuffle and take the first `max_samples` rows.

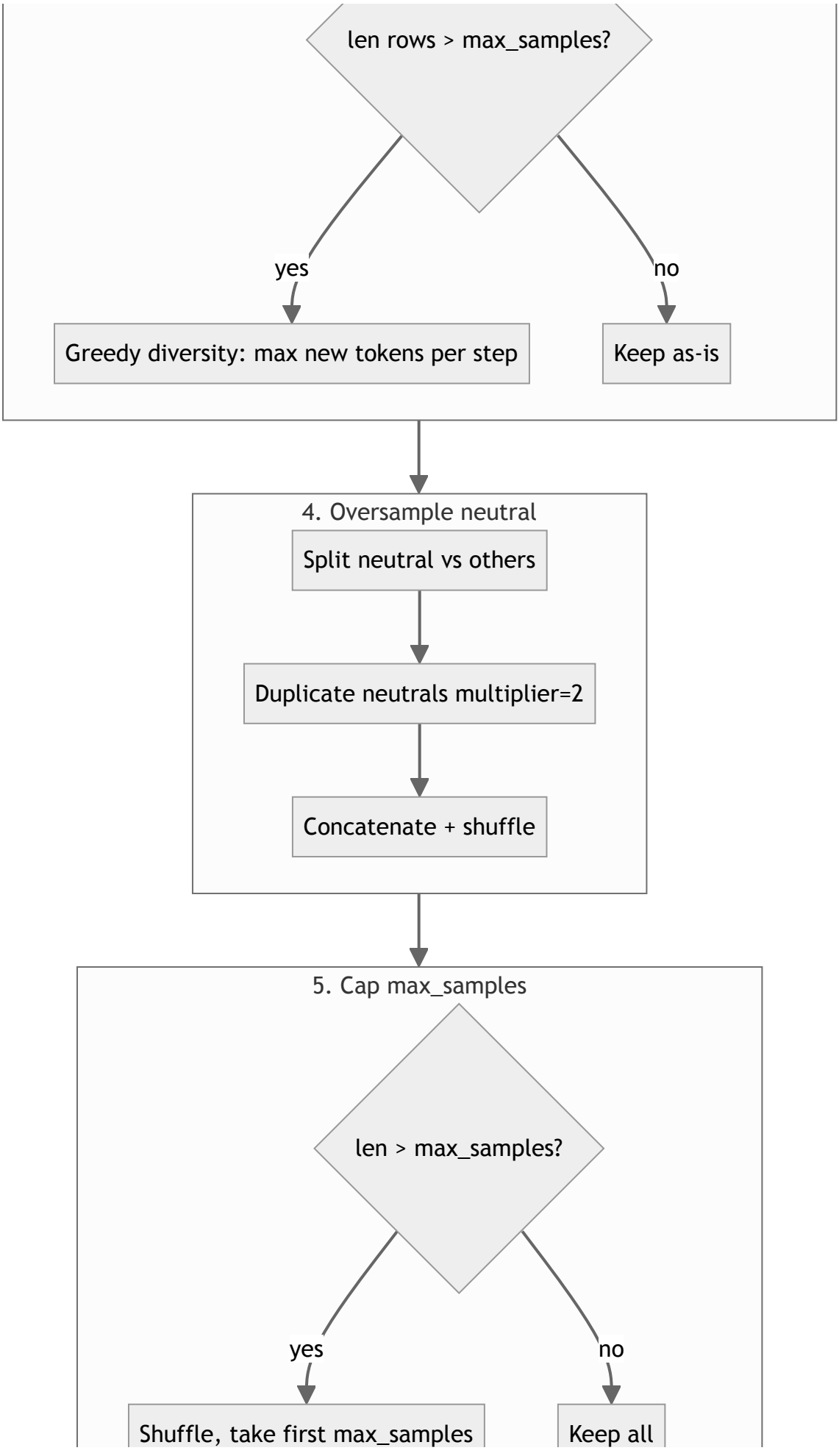
Effect: Final training pool size is at most `max_samples`.

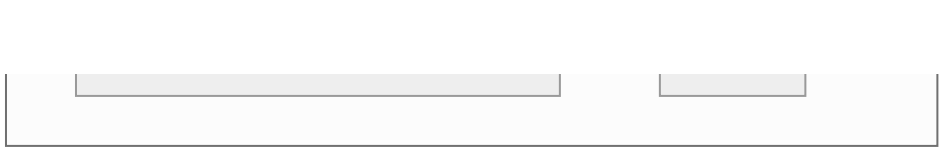
6.1 Order and flow summary

Step	What it does	Typical effect
1. TextBlob	Drop rows where polarity disagrees with rating	Fewer rows; cleaner labels
2. Stratified	Equal count per rating 1–5, up to stratified_max_total	Balanced pool (e.g. up to 10k before cap)
3. VGST	Diversity-based subset toward target size	Runs when pool > cap target
4. Oversample neutral	Double (2×) the number of rating-3 samples	More neutrals; total can exceed max_samples before cap
5. Cap	Shuffle and take first max_samples	Final size ≤ max_samples (default 4000)

6.2 Mermaid: Pipeline step logic (detailed)


1. TextBlob DQC
Compute polarity per review
pol vs rating consistent?
yes or pol=0
no
Keep row
Drop row
2. Stratified
Group by rating 1..5
samples_per_rating = pool÷5 e.g. 400 if pool=2000
Take 400 per rating, shuffle
3. VGST

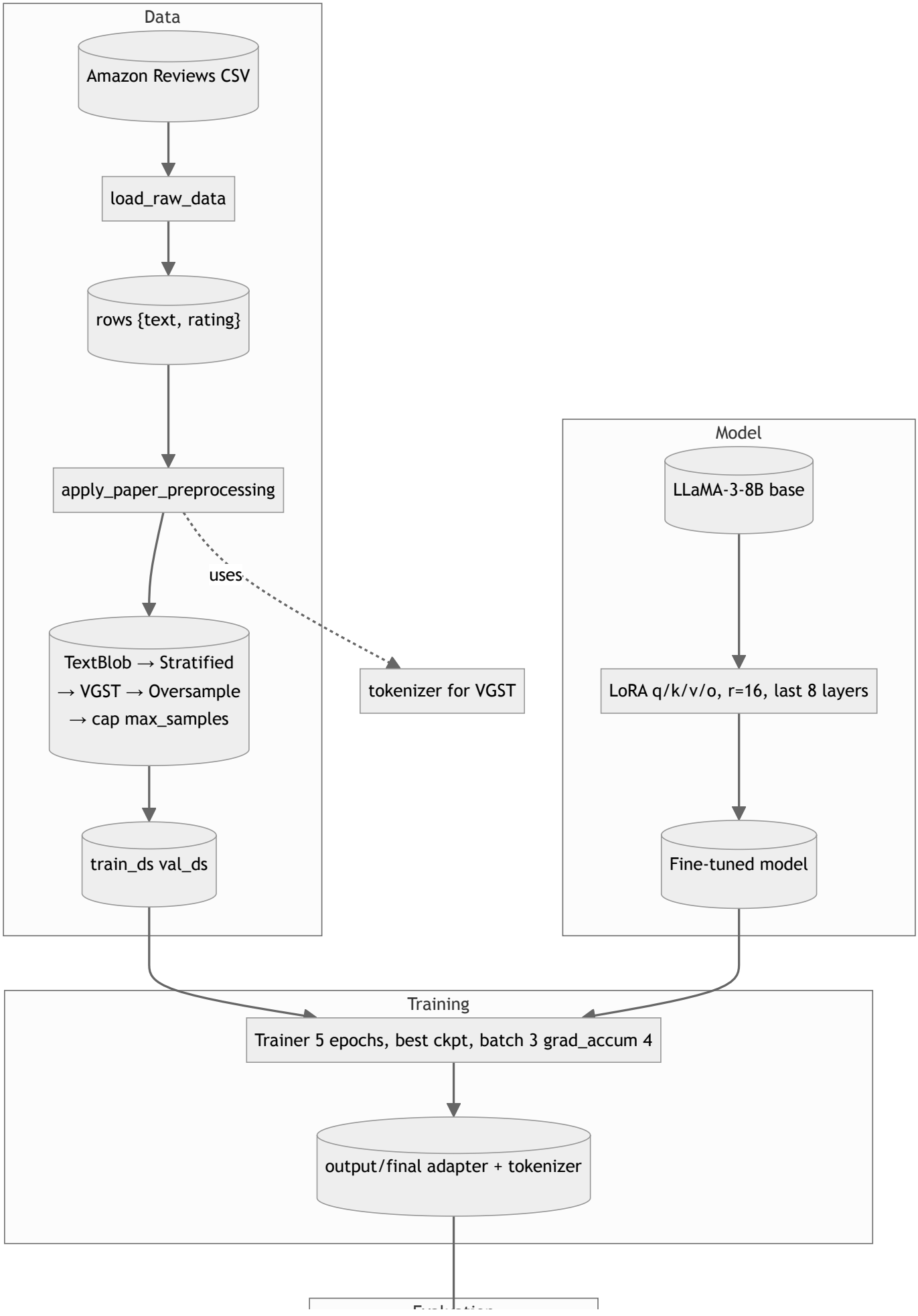

len rows > max_samples?
yes
no
Greedy diversity: max new tokens per step
Keep as-is
4. Oversample neutral
Split neutral vs others
Duplicate neutrals multiplier=2
Concatenate + shuffle
5. Cap max_samples
len > max_samples?
yes
no
Shuffle, take first max_samples
Keep all

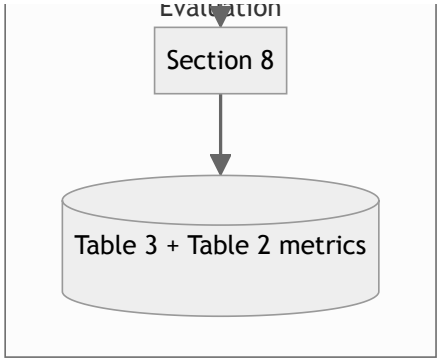


Decision flow within each pipeline step: TextBlob keep/drop; stratified balance; VGST when pool exceeds target; oversample neutrals; cap at `max_samples` (default 4000 in notebook).

7. End-to-End Flow (Data → Model → Training → Eval)

In the notebook, **load_raw_data** reads the Amazon Reviews CSV into rows `{text, rating}` (optional `max_csv_rows`, e.g. 250k). **apply_paper_preprocessing** runs TextBlob → Stratified → VGST → Oversample → cap at `max_samples` (default 4000). The tokenizer is used for VGST. Split: **train_ds** / **val_ds** (e.g. 80/20); validation uses one-shot when enabled (aligned with eval). **Base:** LLaMA-3-8B + **LoRA** on **q/k/v/o**, rank 16, last 8 layers (stronger than paper Table 1). **Trainer:** 5 epochs (default), best checkpoint on `eval_loss`, gradient checkpointing. Adapter in **output/final**. **Section 8** eval: one-shot + CoT, BLEU with correct reference pairing, BLEU-1/chrF++, macro F1.


Data
Amazon Reviews CSV
load_raw_data
rows {text, rating}
apply_paper_preprocessing
uses
TextBlob → Stratified
→ VGST → Oversample
→ cap max_samples
tokenizer for VGST
train_ds val_ds
Model
LLaMA-3-8B base
LoRA q/k/v/o, r=16, last 8 layers
Fine-tuned model
Training
Trainer 5 epochs, best ckpt, batch 3 grad_accum 4
output/final adapter + tokenizer



End-to-end: load CSV, apply preprocessing, split into train/val; base + LoRA → fine-tuned model; Trainer produces adapter; evaluation reports metrics. Preprocessing uses the tokenizer for VGST.

8. Baseline: Task Directive and Rating Scale

Task directive (paper): "Evaluate the sentiment expressed in user reviews and classify each one according to its sentiment rating." **Scale:** Five-point scale: 1–2 negative, 3 neutral, 4–5 positive. "Due to the subtlety in distinguishing neutral reviews, this paper deliberately increased their representation in the dataset."

Rating descriptions (paper wording):

- **Rating 1:** Comments show a high level of dissatisfaction and negativity; serious flaws, unpleasant experiences; strong negative feedback.
- **Rating 2:** Still negative but slightly softened; dissatisfaction plus some areas to improve or unmet expectations.
- **Rating 3:** Generally neutral; both positive and negative aspects; mediocre experience; balanced attitude.
- **Rating 4:** Positive overall; praise with a few shortcomings; generally satisfactory, room for improvement.
- **Rating 5:** High satisfaction; praise and testimonials; little or no mention of deficiencies; willing to reuse or recommend.

One-shot learning: The model is provided with examples of the task (review → rating) so it learns the format and generates appropriate output. Sampled reviews are turned into questions and answers with correct answers; the dataset is in the form of multiple rounds of conversations with many prompts with different expressions. This clarifies task intent and helps avoid overfitting.

Zero-shot Chain-of-Thought (CoT): Used in addition to one-shot to improve reasoning. The model is prompted to reason step by step (thinking chain) from problem to solution. The

paper's implementation adds a directive such as "Let's take it one step at a time" (or a brief overview of ideas) to the prompts.

9. Baseline Model Training (LoRA)

The **base model** is pre-trained LLaMA-3-8B; its parameters are **frozen**. The weight update **ΔW** is not applied in full; it is approximated by two **low-rank matrices B and A**. The formula is `h = Wx + ΔW x = Wx + BAx`. Here **B** is d×r and **A** is r×d with **r ≪ d** (e.g. r = 4); trainable parameters go from d×d to 2·r·d, which greatly reduces memory and training cost. **Initialization:** A from Gaussian, B from zeros, so initial ΔW = 0. **Scaling:** Hyperparameters α and γ; the ratio α/γ scales the update and controls step size (e.g. LoRA alpha = 64).

Key hyperparameters (from paper Table 1):

Parameter	Value	Parameter	Value
learning rate	0.00005	epochs	3
max samples	2000	batch size	3
gradient accumulation	4	cutoff length	1024
LoRA rank	4	LoRA alpha	64
trainable layers	3	val size	0.2
LR scheduler	cosine	optimizer	adamw_torch
compute type	fp16	NEFTune Alpha	0.02

Implementation note: The training notebook may use stronger defaults than Table 1 (e.g. LoRA r=16, α=128, attention q/k/v/o, 8 layers, 5 epochs, `max_samples` =4000) to approach paper-level behavior; reduce settings if GPU OOM.

Training and test data (paper): Table 1 lists 2000 samples; §3 uses 4000 train; 4000 held-out test reviews. Wall-clock depends on GPU.

10. Baseline Results (Our Run vs Paper)

10.1 How we evaluate (current setup)

Evaluation in `colab/llama_sentiment_baseline_train.ipynb` Section 8 matches the paper's **metric types** (Table 3 classification, Table 2 generation/throughput). Goal: **get close to the paper**, not an exact number match.

- **Eval set:** Load raw CSV (up to `max_csv_rows`), exclude rows in `TRAIN_ROWS_SET` when set after training, shuffle (seed 42), take up to `MAX_EVAL_SAMPLES = 4000`. No TextBlob on the eval pool (disjoint from train/val).
- **Inference: One-shot + CoT** via `build_prompt(..., one_shot_example=...)`; `model.generate(..., max_new_tokens=128, do_sample=False)`.
- **Rating extraction:** `extract_rating_from_output` prefers the **last** `Sentiment (1–5):` block (CoT may mention other digits).
- **Table 3:** Accuracy; weighted Precision, Recall, F1; **macro F1** for balanced 5-class view.
- **Table 2:** `sacrebleu.corpus_bleu(decoded_texts, [[r] for r in ref_texts])` — **one reference list per hypothesis**. References are full strings `Sentiment (1–5): {r}. {description}`. Also report BLEU-1 and chrF++ when short outputs make BLEU-4 misleading.
- **ROUGE-1:** Per-sample ROUGE vs full reference string; reported on 0–100 scale in the notebook.
- **Samples/s:** Wall-clock over the eval loop.

Example results (after pipeline updates; your runs may vary) — 4000 eval samples:

Metric	Example run (Colab)	Paper (LLaMA-8B + LoRA)
Accuracy	~0.666 (2664/4000)	0.926
F1 (weighted)	~0.684	0.933
F1 (macro)	~0.499	—
BLEU-4	~60.7	30.23
ROUGE-1	~64.5	85.52
Samples/s	~3.5	6.079

Earlier runs (~59–62% accuracy, BLEU-4 ≈ 0) used older eval code (BLEU reference shape, text-only val, digit-only labels). Re-train with the current notebook for comparable numbers. Paper numbers assume their full experimental setup.

10.2 Improvements already applied (notebook)

- **One-shot at evaluation** and **val** (train-only pool); full-line training targets; stronger LoRA; 5 epochs; best checkpoint; BLEU reference pairing fixed; parser uses last `Sentiment (1–5):`; optional `max_csv_rows` for larger pool.

10.3 Optional next steps

- **Phase 1 vs context-enriched demo:** `colab/phase1_vs_phase2_amazon2023_presentation.ipynb` — same **LLaMA-3-8B + LoRA** adapter; Phase 1-style text-only vs prompt with metadata + optional BLIP caption (not the full agentic stack).
- **Phase 2 LangGraph demo:** `colab/phase2_agentic_vs_phase1_amazon2023.ipynb` — Analyst → metadata grounding → BLIP visual evidence → Critic vs single-pass Phase 1 on Amazon Reviews 2023. Default eval slice is ~**1000** reviews with metadata + product image (checkpoint CSV for resume). Full LanceDB / LLaVA / multi-round reflection remains roadmap (§§12–13).
- **Paper gap:** Exact Table 3 match is not required for the project; document hardware, seeds, and data slice when reporting.

10.4 Amazon Reviews 2023 — Phase 1 vs Phase 2 (implemented Colab eval)

Notebook: `colab/phase2_agentic_vs_phase1_amazon2023.ipynb` in `pes-mtech-project`. It compares the **same** fine-tuned adapter (`output_final.zip` / `output/final`) on a held slice of **McAuley-Lab/Amazon-Reviews-2023** (default category `raw_review_All_Beauty` + `raw_meta_All_Beauty`).

Sample size and filters (default): Target **N_SAMPLES ≈ 1000** review rows after filtering for: non-trivial review text (`MIN_REVIEW_CHARS` , default 20); joined **metadata** with at least title or details; at least one **HTTP product image URL** in metadata (so Phase 2 always has a visual channel to caption). Reviews are drawn in random order (seeded) until the target count or the pool is exhausted.

Data loading (`datasets` ≥ 3): Reviews via `load_dataset("json", data_files=…)` on Hub JSONL; metadata via `load_dataset("parquet", …)` over all shards under `raw_meta_{Category}/` . A single pass builds an ASIN → metadata map (up to `META_INDEX_MAX_ROWS` , default 500k rows) for fast joins.

Multimodal preprocessing: Each row carries `review_text` , `meta_title` , `details` , `main_category` , `image_url` . **BLIP** (`Salesforce/blip-image-captioning-base`) downloads the image and produces `image_caption` used as visual evidence **Ev** in the graph.

Phase 1 (same notebook, same rows): One greedy generation per row — instruction + CoT + one-shot + `Review to classify:` + trailing `Sentiment (1-5):` . **Inputs:** review text only (no metadata or caption in the prompt).

Phase 2 (LangGraph): Analyst — same Phase-1-style prompt on the review (hypothesis / draft rating). **RAG Prover** — non-LLM string from category, title, details (toy **Gf**, stand-in for LanceDB). **Visual Verifier** — non-LLM; injects BLIP caption as **Ev**. **Critic** — second LLM call

with the same LoRA weights; prompt ends with the same training tail (`Review to classify:` … `Sentiment (1–5):`); tokenizer may use `truncation_side="left"` so the tail is preserved under long context. **Generation:** explicit `GenerationConfig(do_sample=False, …)` to avoid invalid merged `temperature` / `top_p` warnings.

Scoring: Exact-match **accuracy** and **MAE** on star labels (1–5) vs review `rating` as ground truth. **LLM calls per row in this comparison:** Phase 1 → 1; Phase 2 graph → 2 (Analyst + Critic), plus the separate Phase 1 call in the eval loop → **3** forward generations per row total for the side-by-side table.

Scale and checkpoints: Full 1000×3 generations is GPU-heavy (many hours on a T4). Results stream to `phase1_vs_phase2_amazon2023_results.csv` (override with env `RESULTS_CSV`) every `CHECKPOINT_EVERY` rows (default 25); re-running the eval cell **resumes** using column `sample_idx` . Plots: accuracy/MAE bars plus a short prefix line chart of per-sample ratings.

Rating parser: Decoded text is often only the continuation after `Sentiment (1–5):` ; extraction handles leading `5.` / `5 …` and full-line `Sentiment (1–5):` patterns, not only substrings in the completion.

Related notebook: `phase1_vs_phase2_amazon2023_presentation.ipynb` compares Phase-1-style text-only vs a **single** enriched prompt (metadata + optional BLIP) without the LangGraph Critic — useful ablation, not identical to Phase 2.

Paper comparison with other baselines (Table 3): Decision Tree (CountVectorizer): Accuracy 0.786, F1 0.974; SVM (TfidfVectorizer): 0.872, 0.874; Multinomial Naïve Bayes: 0.876, 0.882; XLNet-Large-Cased: 0.916, 0.920. The proposed LLaMA-8B + LoRA outperforms all of these.
Metric meanings: BLEU-4 measures 4-gram overlap between generated and reference text (fluency/grammar); ROUGE-1 measures unigram overlap (lexical recall); samples per second measures throughput.

11. Resources & Why Colab

Resources used to run this:

Resource	What the run uses
GPU	NVIDIA GPU with ≥16 GB VRAM (e.g. Colab T4 16 GB). LLaMA-3-8B in fp16 needs ~16 GB; in 4-bit it fits in ~8–10 GB, so a T4 is enough.
4-bit quantization	bitsandbytes (NF4, fp16 compute). Without it, full fp16 needs ~16 GB VRAM; with 4-bit, training fits on a single T4.
RAM	Several GB for data load (50k–500k rows CSV), tokenization, and PyTorch. Colab gives ~12 GB RAM.
Batch size	3 per device, gradient accumulation 4 → effective batch 12. Kept small to avoid OOM on 16 GB.
Training time	Paper: ~50 min/epoch on RTX 4090 with 5k rows. On Colab T4 with 2000 samples, ~10–15 min/epoch typical (3 epochs → ~30–45 min).
Disk	Model + adapter + tokenizer: a few GB. Colab runtime has limited disk; output is zipped and downloaded or copied to Drive.

Why it often doesn't work locally:

1. **No GPU or small GPU:** LLaMA-3-8B in fp16 needs ~16 GB VRAM. Many laptops have no discrete GPU or only 4–8 GB. Without 4-bit, training runs out of memory (OOM).
2. **4-bit (bitsandbytes) is Linux/CUDA-only for training:** bitsandbytes is built for Linux + NVIDIA CUDA. On macOS or Windows it often doesn't install or load correctly, so 4-bit fails and you fall back to fp16 → OOM.
3. **CUDA and drivers:** Local runs need NVIDIA drivers and CUDA and a compatible PyTorch build. Colab provides this; on Mac (M1/M2/M3) there is no NVIDIA GPU.
4. **Environment and dependency order:** The notebook recommends restarting the session after certain cells so that 4-bit loads correctly. Locally, wrong Python env or import order can make 4-bit "skip" and trigger OOM.
5. **Local repo:** The repo's Makefile and train.py still require a suitable NVIDIA GPU + CUDA + bitsandbytes. On a Mac or machine without a 16 GB (or 4-bit-capable) GPU, training will not work the same way as on Colab.

Why we use Colab:

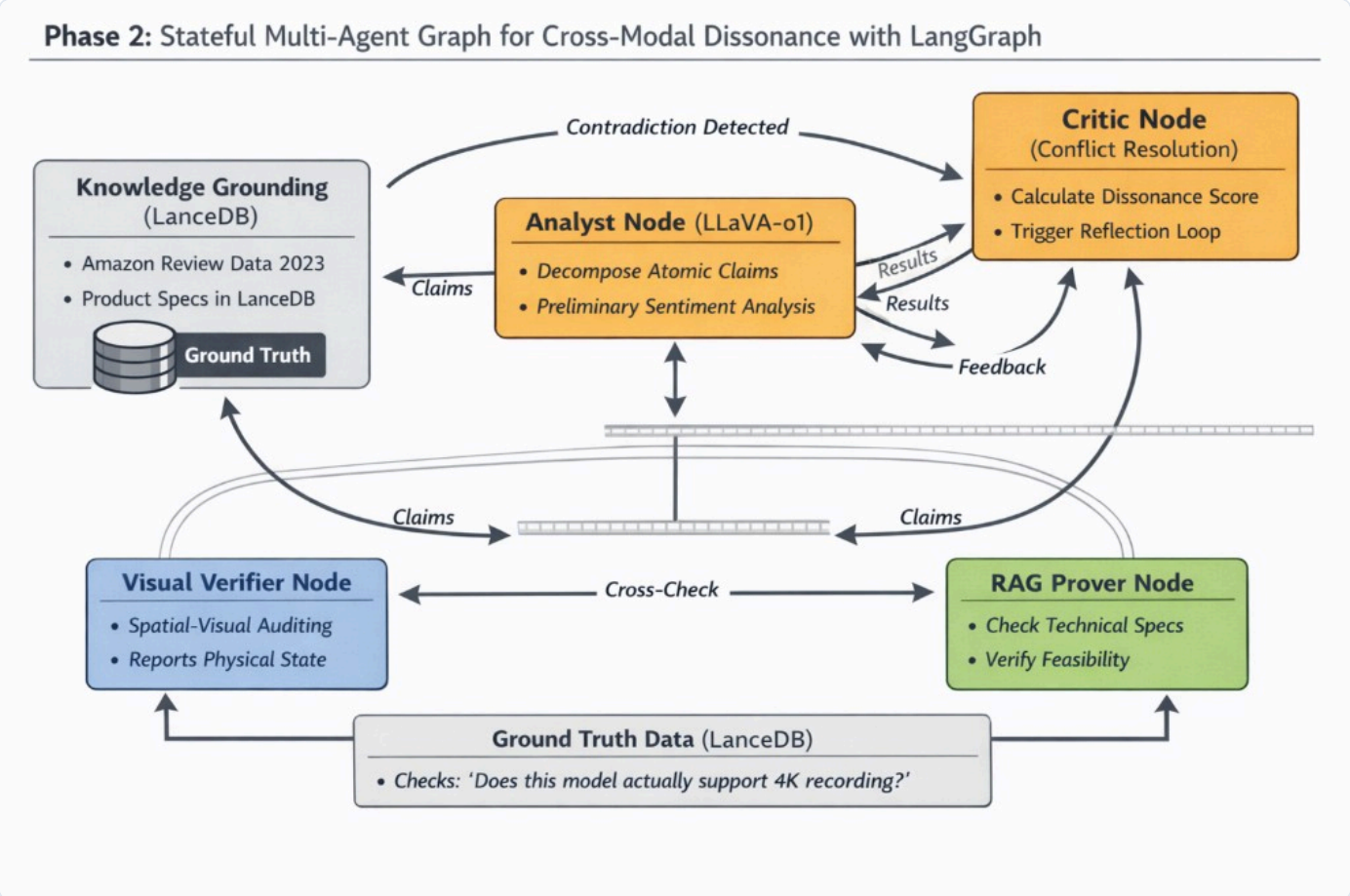
Reason	Explanation
Free GPU	Colab (free tier) gives an NVIDIA T4 16 GB. Enough for 4-bit LLaMA-3-8B + LoRA training.
Correct environment	Linux + CUDA + bitsandbytes are already set up. 4-bit loads reliably after “Restart session, Run all.”
No local GPU needed	Runs in the cloud; your machine only needs a browser.
Reproducibility	Same notebook, same runtime type (e.g. T4) → similar resource usage for everyone.
One place for data + model + run	Dataset (Kaggle), model (Hugging Face), and run are all in one environment.

When local can work: If you have a Linux (or WSL2) machine with an NVIDIA GPU (≥16 GB) and install CUDA, PyTorch, and bitsandbytes correctly, you can run the same pipeline locally (e.g. the repo’s train.py or the notebook with a Jupyter kernel that sees the GPU). macOS (Apple Silicon) and Windows without a proper CUDA GPU are not supported for this 4-bit training path.

12. Proposed System: Phase 2 Multi-Agent Architecture

The proposed framework moves away from single-inference classification and adopts a **Stateful, Multi-Agent Workflow** orchestrated by **LangGraph**. Inputs include review text, product metadata, and product images. The **Analyst** (LLaVA-o1) processes the review and decomposes atomic claims, producing a preliminary sentiment and a **hypothesis vector H**. Claims are sent to the **Visual Verifier** (spatial auditing on images; evidence vector **Ev**) and the **RAG Prover** (queries LanceDB for specs; factual grounding score **Gf**). The **Critic** compares H with Ev and Gf, computes **dissonance D = Norm(H – (Ev + Gf))**, and either releases the verified sentiment (low D) or triggers a **reflection loop** back to the Analyst with feedback (high D). **Knowledge Grounding (LanceDB)** is the non-parametric source of truth (CLIP vectors + schema-rich metadata).

Image path: Open this HTML from the project root so the diagram loads, or set the path below to match your folder layout.



13. Proposed System: Key Nodes (Detailed)

13.1 Knowledge Grounding (LanceDB)

Acts as the system's **"Source of Truth"** or non-parametric memory. It stores Amazon Review Data 2023 and product specs. **Vectorization:** CLIP (ViT-L/14) creates a shared space where product images and technical text specifications are comparable. **Schema-rich metadata:** Not just text—structured tensors for hardware SKUs, port types, dimensions. **Significance:** The system can be updated with new 2024–2026 product data instantly without retraining the underlying LLaVA or LLaMA models.

13.2 Analyst Node (LLaVA-o1)

The **"First Responder."** Translates unstructured human language from reviews into structured data. **Tasks:** Decompose atomic technical claims (e.g. "Screen brightness is low") and perform preliminary sentiment analysis. **Output:** A **Hypothesis Vector (H)** containing preliminary sentiment and identifying which aspects need factual verification by the Verifier and Prover.

13.3 Visual Verifier Node

Responsible for **spatial-visual auditing** and reporting physical state. It addresses the **"Semantic Leakage"** problem (where the AI "sees" what it "reads") by being strictly isolated: prompted to act as a "Hardware Auditor," **ignoring the user's emotional tone**. It performs patch-level analysis on specific coordinates of product images to confirm physical states (e.g. verifying a cracked screen or missing port). **Output: Evidence Vector (Ev)** with raw physical facts.

13.4 RAG Prover Node

Acts as the **"Product Specialist"** by querying the LanceDB specification store. **Tasks:** Check technical specs and verify feasibility. It identifies **User Expectation Mismatches**: e.g. a user gives 1 star because "The USB-C cable doesn't fit the port," and the SKU spec confirms the device only has Micro-USB—the Prover flags this as user error, not product defect. **Output: Factual Grounding Score (Gf)**.

13.5 Critic Node & Reflection Edge

The **"Brain"** that manages flow and resolves conflicts. **Dissonance calculation:** D = Norm(H – (Ev + Gf)). The **Reflection Edge (the loop):** If **D < threshold**, data is consistent and the verified sentiment is released. If **D > threshold**, the Critic rejects the Analyst's report and sends it back with specific feedback (e.g. "Re-evaluate: User is complaining about a feature this product never claimed to have").

14. Evaluation Metrics (Phase 1 vs Phase 2)

Phase 1 (baseline) uses classification and linguistic metrics; Phase 2 adds grounding, verification, and explainability metrics to measure factual correctness and the cost of the reflection loop.

Implemented Amazon 2023 slice (§10.4): For the LangGraph demo, reported numbers are **star accuracy and MAE** on the filtered ~1000-row slice (not the full Table 3 BLEU/ROUGE suite). Fill in concrete Phase 1 vs Phase 2 accuracy/MAE from your finished `RESULTS_CSV` after the long eval completes.

Category	Metric	Phase 1	Phase 2	Formula / Logic
Classification	F1	✓	✓	F1 = 2·P·R/(P+R); balanced across 5 star-rating tiers
Reliability	Precision P	✓	✓	TP/(TP+FP); reduction in false positives (user errors labeled as defects)
Grounding	Faithfulness	—	✓	Claims supported by LanceDB / Total claims; RAGAS framework
Verification	Dissonance detection	—	✓	Accuracy of Critic in identifying mismatches between user text and evidence
Logical	Dissonance score D	—	✓	D < 0.3; distance between Analyst claim vector and Verifier/Prover evidence
Linguistic	BLEU-4, ROUGE-1	✓	✓	4-gram precision; unigram recall of keywords in explanation
Explainability	Traceability depth	—	✓	Avg reflection edges per input; “System 2” effort
Spatial	Visual IoU	—	✓	Overlap/Union for Visual Verifier’s localization of product parts
Efficiency	Inference speed	✓	✓	Samples per second (e.g. on RTX 4090)

15. Summary

Baseline (Phase 1): LLaMA-3-8B + LoRA on Kaggle Amazon-style reviews; TextBlob DQC, stratified pool, VGST, oversample neutral, cap `max_samples` (default 4000); 5 epochs, best

checkpoint; One-Shot + CoT; eval aligned with training. Paper-reported ~92.6% accuracy is a reference; our implementation targets **the same methodology and ballpark metrics**.

Proposed (Phase 2): Multimodal agentic framework with Knowledge Grounding (LanceDB), Analyst (LLaVA-o1), Visual Verifier, RAG Prover, and Critic; Verifier-in-the-Loop with dissonance calculation and reflection. The goal is sentiment predictions that are both **linguistically accurate** and **factually grounded** for e-commerce, reducing sentiment hallucinations and improving reliability of insights from customer feedback.

Colab realization today: A LangGraph pipeline with the **same LoRA as Phase 1** plus metadata + BLIP captions reconciled by a Critic is evaluated on ~1000 Amazon Reviews 2023 rows (§10.4); production LanceDB/LLaVA features remain future work.

16. References

References used for this solution (baseline methodology and proposed Phase 2 architecture).

16.1 Self-reflection and Critic / Reflector

- **Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection.** Introduces a "Reflector" agent that generates personalized reflections on agent performance to catch errors and improve task execution; directly analogous to our **Critic Agent**. arXiv:2310.11511.

16.2 Cross-modal verification and Verifier

- **MERMAID: Multi-perspective Self-reflective Agents with Generative Augmentation for Emotion Recognition** (ACL Anthology 2025). Uses a **Cross-Modal Verification** module structurally similar to our **Visual Verifier** that cross-checks sentiment against specifications; reports absolute accuracy gains of up to 27% from self-reflection loops. ACL Anthology 2025.emnlp-main.1252.

16.3 Verifier-in-the-loop

- **Local Look-Ahead Guidance via Verifier-in-the-Loop for Automated Theorem Proving** (Findings of ACL 2025 / ICLR 2025 Workshop). Popularizes the term **"Verifier-in-the-Loop"**: an external verifier provides real-time feedback to a generator model. Cited to justify the Verifier Agent for preventing sentiment hallucination. arXiv:2503.09730.

16.4 Agentic evaluation in e-commerce

- **Can LLM Agents Simulate Customers to Evaluate Agentic-AI-based Shopping Assistants?** (Sun et al., 2025). Discusses agents as "Digital Twins" for evaluation; relevant for Phase 2 evaluation (e.g. simulating customer personas to stress-test sentiment analysis). arXiv:2509.21501.

16.5 Dataset

- **Amazon Reviews 2023** (UCSD McAuley Lab). Multimodal dataset (text, metadata, images) for Phase 2. amazon-reviews-2023.github.io.

16.6 Other references

- SSRN abstract 5351494.

- IEEE Xplore document 10938581.

17. Implementation Status — What We Implemented So Far

Phase 1 baseline training/eval is implemented in `colab/llama_sentiment_baseline_train.ipynb`. **Phase 2** production stack (§§12–13) remains **proposed**; a **Colab LangGraph demo** (`colab/phase2_agentic_vs_phase1_amazon2023.ipynb`) implements Analyst → metadata → BLIP → Critic vs Phase 1 on ~1000 filtered multimodal rows, with **resumable CSV checkpoints** (§10.4). The **context ablation** notebook (`colab/phase1_vs_phase2_amazon2023_presentation.ipynb`) compares text-only vs enriched single prompt on the same adapter. Amazon Reviews 2023 in those notebooks uses **JSONL + Parquet** loading (§4.2) for `datasets` 3.x.

17.1 Implemented (Phase 1 baseline)

Preprocessing: TextBlob DQC → stratified (large pool, e.g. 10k) → VGST → oversample neutral → cap `max_samples` (default 4000); optional `max_csv_rows` for ingest. See `apply_paper_preprocessing()` in `colab/llama_sentiment_baseline_train.ipynb`.

- **Prompts / SFT:** Instruction, CoT, one-shot; training target = full rating line; val uses one-shot when enabled.
- **Model:** LLaMA-3-8B, 4-bit optional; LoRA on q/k/v/o, r=16, α=128, last 8 layers; 5 epochs; best checkpoint; gradient checkpointing.
- **Eval (§8):** One-shot + CoT; BLEU with `[[r] for r in ref_texts]`; BLEU-1/chrF++; macro F1; parser prefers last `Sentiment (1-5):`.
- **Deliverables:** Adapter + tokenizer in `output/final`; zip `output_final.zip`; docs in repo root (e.g. `CODE_EXPLANATION.md`, `PROJECT_CHANGES_SUMMARY.md`).

17.2 Example metrics

Kaggle baseline (§10 table): Typical post-update run ~66% accuracy on 4000 held-out eval rows; varies by seed. Goal: **close to paper methodology**, not identical.

Amazon Reviews 2023 agentic eval (§10.4): After running `phase2_agentic_vs_phase1_amazon2023.ipynb` to completion, read accuracy/MAE from the notebook output or aggregate `phase1`, `phase2`, and `gt` in the checkpoint CSV.

17.3 Not yet implemented

- **Phase 2 production system:** LanceDB + CLIP store, LLaVA-o1-class Analyst, full multi-round reflection and Verifier-in-the-Loop at scale (beyond the Colab LangGraph skeleton).
- **Optional:** Table 3 classical baselines; Table 2 other LLMs; full 500k CSV if RAM allows.